syntax error
Logical error
Runtime error

```
Cout<<"plz,enter your Name:"<<endl;
String n;
cin>>n;
Cout<<"plz,enter your birth year:";
int y;
cin>>y;
Cout<<"Hello";
Cout<<n;
Cout<<",Your Age is";
Cout<<2025-y<<endl;
```

```
int age=2025-y;
Cout<<age<<endl;
```

```
Cout<<"hello"<<n<<",Your age is"<<2025-y<<endl;
```

```
plz,enter your Name:
Ali
plz,enter your birth year: 2003
Hello Ali, your age is 22
```

C++

- ✓ Microsoft Visual Studio
- ✓ Errors
- ✓ output •
- Input •
- ✓ variables •
- Processing •
  - ✓ Arithmetic operations
    + - * / %

		Data Type
"Ahmed"	text →	String
25	Integer →	int
3.55	Floating Point No →	double
'A'	Character →	char

Variable Definition

DataType variablename;

String email;

int age;

double gpa;

char x;

syntax error
Logical error
Runtime error

Cout << ;

cin >>

```
String name;
cin>>name;
Cout << "Hello";
Cout << name;
```

Ahmed ↓

Hello Ahmed

C++

✓ Microsoft Visual Studio

✓ Errors

✓ output •

- input •

✓ Variables •

- Processing •

✓ Arithmetic operations

+ - * / %

character ✓ → int age; ✓ string x; ✓

symbols X → String #name; X int age$; X double x$5; X

underscore ✓ → String _email; ✓ string first_name; ✓ String first_name; X

Number X → char 3rnr; X int n1; ✓

Reserved Keyword X → String Cout; X int double; X String Cout; ✓

Cout << ( ) ;

27%4 = 3

⓪ ()

① * / %

② + -

→ Text "hello"

→ Number 2025

→ Expression Result: 5+3*2

→ Variable

5+3*7/3%3+5

21/3

7%3

5+1+5

6+5

11

Cout<<

3*(5+12/6*2%(5-6));

45+12/6*2%5-6,

45+2*2%5-6

45+4%5-6

45+4-6

49-6

43

3*27/6*2%(-1)

81/6*2%-1

13*2%-1

27%-1

27

syntax error

Logical error

Runtime error

Cout<< 5*2;

5+3;

5-4;

6/3;

28%5;

3

- create empty project
- Add new file (CPP)
- Write fixed code
- Build project
- Run project

```
#include <iostream>
using namespace std;
void main()
{

    system("Pause");
}
```

Pressing Key to continue

C++

✓ Microsoft Visual Studio

✓ Errors

- Output •
- Input •
- Variables
- Processing •
  - Arithmetic operations
    + - * / %

Chapter Six

C++ Program Structure

Consider the following simple C++ program:

```
// this program computes the sum of two integer constants, and
assign the result to an integers      // variable, then displays it.
#include<iostream.h>
void main( )
{
        int sum ;
        sum = 10+ 20 ;
        cout <<"the sum of 10 and 20 is "<< sum<<"\n" ;
}
```

A C++ program is composed of one or more functions. Each function is identified by its name. the name of the function is chosen by the programmer. The exception to this rule is that there must exist one and only one function called **main** in any given program.

The program starts executing from the first instruction in the function main. This function executes instruction and invokes other functions to perform their operations. The other functions may be part of the same program or be members of libraries of previously developed functions. The standard library is a collection of C++ functions designed to provide, among other things, standard I/O facilities.

6.1 The program statements:

cout<< " the sum of 10 and 20 is "<<sum<<"\n" ; deliver output to the user's terminal.

Output is directed to the user's terminal, referred to as the standard output, and associates with the predefined iostream (I/O stream)

cout, which is defined in the header file iostream.h. The #include line is a pre-processor directive, and causes the content of the named file to be read into the program text file when its compiled, from which we obtain the details about cout. The output operator (<<), referred to as insertion, is implement as a standard library function. The body of a function consists of definitions and statements enclosed within the braces { and }.

In C++, every statement must have a terminating semicolon.

In the output statement, shown in the example program, the sequence \n is the C++ notation for a new line symbol. When a new line is printed, all subsequent output starts from the left margin of the next line.

The statement **int sum ;** is an example of a variable definition. The variable sum is declared to be of type int (integer).

Besides whitespace (blank space, line separators, and tab characters), the above program uses a variety of punctuation

Symbols :

- Braces { and }, which have been noted as function body delimiters.
- Double quote (") character, which are used to delimit character string .
- The semicolon, the statement terminator.

Other punctuation symbols used with C++ programs are :

- The comma (to separate items in a list), and
- Parentheses (and) to bracket one or more items.

In the above program, we see the word **void**. It is a C++ reserved keyword which indicates that the function main dose not return any value.

Finally, the example program includes comments which are preceded by // symbols.

Note: in C++, use in made of both upper and lower case letters to name program objects. The name **main** in the above program must appear as shown. The names MAIN and Main, for example, would be interpreted as different names from their lower case form.

Compiling and running C++ programs:

The source program is placed in a file which must have the extension .cpp. The C++ compiler translates the source program into its equivalent machine instructions. During the translation process, the compiler may detect errors in the programming language usage, for example a missing comma or semicolon. The compiler reports these errors in a special file showing the source and nature of the error. These are known as **compile time errors.**

the executable program file is produced by combining or linking the compiled image of the program with any parts required from libraries. If any necessary parts are not found in the libraries then a link error occurs.

If the program compiles and links correctly, it can be executed. When running this executable program, further situations, known as run time errors may occur. These cause programs to terminate abnormally during execution. Example of such errors are division by zero, and arithmetic overflow.

A program that is free of both compile time and run time errors may still not produce the correct result. These errors, known as logical errors , arise from an incorrect specification or implementation of the program solution. This type of errors can be detected by running

the program with test data, and comparing the result with the expected ones.

6.2 C++ Programming

C++, an object oriented extension of C, addresses the needs of two recent trends: the widespread use of C as a software implementation language, and the increasing interest in object-oriented programming.

6.3 Elements of C++.

The basic C++ constructions: constants , variable declarations, arithmetic expressions, function calls and definitions, and input, output, and control statements, are given in the following sections.

6.3.1 Comments:

Comments can be presented with a pair of leading // symbols. The remainder of the line following the // is considered comment and is ignored by the compiler. The comment symbols can appear anywhere in a program and are often appended to a program statement. Examples of this comment style are:

```
// this is a comment
int sum ; // declare an integer variable
```

A second comment style is any sequence of characters enclosed between /* and */. Such a comment can extended over any number of lines. An example of this style of comment is :

```
/* Now in the time
   For all good programmers
   to put comment in their programs */
```

6.3.2 Constants:

Integer constants: An integer constant is a whole number which may be positive, zero or negative. Integer constants may be specified

"This string terminates with a newline \n"

A string is allowed to extend over one or more lines provided each line terminates with a backslash character followed immediately by a newline symbol. In this case, the backslash and the newline symbols are ignored. For example, the string :

" This string extends \
over two lines."

is textually equivalent to :

"This string extends over two lines."

Two adjacent string constants are concatenated at compile time into a single string. For example: " This" "string" is converted into : "This string".

This concatenation is convenient for splitting long strings across several source lines.

For each string constant of N characters, there is a block of N+1 consecutive characters whose first N are initialized with the characters from the string, and whose last character is the ASCII NULL character '\0'. Thus the string: "hello" is stored as shown below:

h	e	l	l	o	\0

It should be noted that the character 'x' and the string "x" are not the same. The character 'x' is a single character 8-bit byte whilst the string "x" is two characters bytes as shown below :

x

The character 'x'

x	\0

the string "x"

6.3.3 Identifiers:

The C++ programming language requires that various quantities used in a program be given names by which they can them

be referenced. These names are known as **identifiers** and are created by the programmer.

An identifier is created according to the following rule:

- An identifier is a combination of letters and digits the first of which must be a letter. The underscore symbol (_) is permitted in an identifier and is considered to be a letter.

An identifier may not be the same as any of the C++ keyword which are listed below:

Asm	else	private	throw
Auto	enum	protected	try
Break	extern	public	union
Case	float	register	unsigned
Catch	for	return	virtual
Char	friend	short	volatile
Class	goto	signed	wchar_t
Const	if	sizeof	while
Continue	inline	static	
Default	int	struct	
Delete	long	switch	
Do	new	template	
Double	operator	this	

Some valid identifiers are: count COLUMN25 get_char _dos_call

Some invalid identifier are :

1234c hello.cpp $amount id_# get-char if

The first of these begins with a digit, the next four contain illegal characters, and if is a keyword. C++ identifiers are case sensitive , so that name, Name, and NAME are three different identifiers. Identifiers containing leading pairs of underscores (such as _ _MAX) are reserved for use by C++ implementations and standard libraries and must be avoided by users.

The name of the identifier should be reflect the purpose of the object in a C++ program.

6.3.4 Variable declaration

Variables are data items whose stored values vary during the execution of the program. Variables must be declared before they are used. Variables declaration takes the form:

type-specifier list-of-variables;

The list-of-variables is a comma separated list of identifiers representing the program variables. Each named variable possesses as fundamental type as given by the type-specifier.

The type-specifiers for the five fundamental types are introduced by the reserved keywords :

int	(integer)
long	(long integer)
char	(character)
float	(single precision floating point)
double	(double precision floating point)

Following are example of variables declarations:

```
int  hours, minutes, seconds ;   // 3 variables of type int
float centigrade, Fahrenheit ;   //2 variables of type float
char initial ;                   // a variable of type char
double epsilon ;                 // a variable of type double
```

long total ; // a variable of type long

The first declaration means, the variables hours, minutes, seconds may assume any legal integer value, and integer values may be assigned to these variables.

Normally, a variable declaration both declares an identifier (states its properties to the compiler) and defines it (creates the region of memory in which the value of the variable can be stored). Thus variable declarations could just as well be called definitions

A variable can be assigned an initial value when it is declare. For example, the declaration and initialization int n = 25; declares the variable n and initializes it to 25. Likewise, double p = 7.8, q = 4.9, r = 25.35 ; declares the variables p, q, and r and assigns them the initial values 7.8, 4.9, and 25.35, respectively.

Declarations can be intermixed freely with other statement they don't have to be grouped at the beginning of a block. Rather than declaring a variable at the beginning of a block float w; and assigning it a value later, w = 3.1416F;

We instead declare and initialize the variable at the point where the first assignment takes place.

Float w = 3.1416 F;

Declaring long, short, and unsigned integer variables:

- If the qualifier long is placed before the type specifier int in a declaration, then the integer variables will have extended range. Long integer variables provide the largest range of signed integer values. An example of a long definition is: long int memory_address, factorial ;
- The qualifier short placed a head of the type specifier int in a variable declaration specifies that the named variables are used to represent

```
const double  pi = 3.1415926 ;
const int     dozen = 12 ;
```

6.3.5 Arithmetic expressions:

The C++ programming language supports the normal arithmetic operators : addition (+) , subtraction (-), multiplication (*) and division (/). Also provided is the modulus operators (%) used to compute the reminder on dividing two integer values. These are the binary arithmetic operator which apply to two operands. The operators + and – are unary when they take only one operand, as in +x or –m*n.

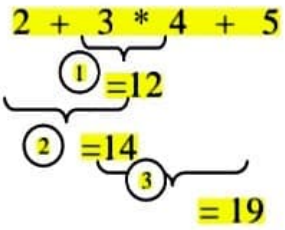
Examples of expressions involving such operators are:

```
12 * feet
-1023
3.14 * radius * radius
distance / time
cents % 100
```

As with normal arithmetic expressions, a C++ expression is evaluated according to the precedence of its operators. The precedence of the arithmetic operators, from highest to lowest, is as follows:

```
* , / , %
+ , -
```

The expression 2+3*4+5 is evaluated as follows:



If, in the above expression, it is required to perform both the additions before executing the multiplication, then this is indicated by employing the parentheses (and) around the sub expressions. The expression would then be written as (2 + 3) * (4 + 5), and evaluate to 5 * 9, or 45.

For two integer operands, the division operator determines the quotient with any fractional reminder truncated.

Thus :

13.0/5	evaluates to 2.6
13/5.0	evaluates to 2.6
13/5	evaluates to 2

The modulus operator (%) yields the integer reminder from the division of two integer operands. Therefore:

13%5	evaluates to 3
15%5	evaluates to 0

If either operand is negative, the reminder usually has the same sign as the dividend. Thus:

-13%5	evaluates to - 3
15% -5	evaluates to 3

6.3.6 assignment operators

The assignment operator allows the assignment of some new value to a variable. The simplest form of the assignment operator is:

Variable = expression

The effect of the operator is to evaluate the expression to the right of the assignment operator (=), and the resulting value is then assigned to the variable on the left.

If both operands of the assignment operator are arithmetic, the type of the expression is converted to the type of the variable on the left before executing the assignment. Examples of assignments are:

interest = principal * rate * time /100

count = count +1

The assignment operator has a very low precedence, and right to left associatively.

The low priority of the assignment operator guarantees that the right hand expression is first evaluated before the assignment is performed.

As assignment, such as n = 10, is an expression and yields a value just like any other expression; the value returned is the same as the value assign to the variable.

The statements m = n = 10; first assigns 10 to n; the value of the assignment expression, 10, is then assigned to m. Thus both are assigned the value 10.

The statement m = n = 10; is equivalent to m = (n = 10); the parentheses are unnecessary because of the right to left associativity of the assignment operator.

Likewise ,

X = y = z = p + q;

Is equivalent to

x= (y = (z = p + q));

<u>The compound assignment operators:</u>

Assignments of the form:

preserved for succeeding execution of the block. Data object with these properties are said to have storage class **automatic**.

The static storage class will be discussed later.

6.3.10 Input and Output

The C++ library provide very convenient facilities for input and output. In C++ , input is read from and output is written to streams. When iostream.h is included in a program, several standard streams are defined automatically.

The stream Cin is used for input, which is normally read from the user's keyboard .

The stream cout is used for output, which is normally send to the user's display.

The insersion operator , << , inserts data into a stream, thus the statement

```
Cout<<500;
```

Places the value 500 in the standard output steam cout, and will be printed on the user's display.

A series of << operators can be used to output several data values with a single statement. For example,

```
Cout<<500<<600<<700;
```

Output the three numbers in the form : 500600700

The statement

```
Cout<<" this is a string ";
```

Prints the string literal *this is a string*

The extraction operator, >> , fetches data from a specified input stream, for example, using the predefined input stream cin, we can direct the input from the keyboard to a variable X as follows :

```
cin>> X;
```


a series of >> operators can be used to read several data values with a single statement , as in the following example :

```
cout<<"please enter 2 numbers:";
cin>> first >>second;
```